

Electronic Scale Design, Test, and Analysis: Based on CZL611N Load Cell and HX711 24-bit ADC

Zhuorui YUN

Department of Mechanical Engineering and Robotic

Guangdong Technion – Israel Institute of Technology

Portfolio version

May 2026

Abstract

This report documents the design, calibration, signal processing, and performance evaluation of a precision electronic weighing scale built around the CZL611N strain-gauge load cell and the HX711 24-bit analog-to-digital converter (ADC), driven by an Arduino Mega microcontroller. The system targets a static measurement accuracy of better than 0.1 g over a 0 g–1000 g full-scale range with a minimum update rate of 10 Hz.

Hardware construction, a two-stage polynomial calibration pipeline, a two-stage digital filter (5-sample median followed by EMA), an auto-zero startup routine, and real-time web-based monitoring were all implemented and evaluated. The calibration pipeline applies a primary 4th-order nonlinearity correction followed by a secondary 10-point interactive precision correction, achieving an RMSE of 0.049 g and a maximum absolute error of 0.098 g across the 0 g to 1000 g range. The two-stage filter (median window = 5, EMA $\alpha = 0.23$) reduced output noise by 18.2%, from $\sigma = 0.092$ g (raw) to $\sigma = 0.075$ g (filtered), with no deadband discontinuity. The measured 10%–90% rise time on a 500 g step was 4.45 s and the 2% settling time was 7.72 s, dominated by HX711 internal filtering and load-cell mechanical dynamics. A browser-based web dashboard replaces a conventional LCD display, and a buzzer overload alarm provides audible protection at 950 g.

Keywords: load cell, HX711, polynomial calibration, median filter, exponential moving average, zero-drift, WebSocket, Arduino.

Contents

1	System Overview	4
1.1	Hardware Architecture	4
1.2	Wiring Summary	4
1.3	Software Architecture	5
2	Calibration	5
2.1	Procedure	5
2.2	Polynomial Fitting and Order Selection	6
2.3	Selected Calibration Model	6
2.4	Two-Stage Calibration Pipeline	7
3	Calibration Validation Results	8
3.1	Reference Data and Measurement Protocol	8
3.2	Validation Table	9
3.3	Discussion of Validation Results	9
4	Signal Processing and Filtering	9
4.1	Filter Architecture Overview	9
4.2	Exponential Moving Average (EMA)	10
4.3	Auto-Zero at Startup	10
4.4	Firmware Implementation	11
5	Zero-Drift and Noise Characterisation	11
5.1	Experimental Procedure	11
5.2	Results	11
6	Dynamic Performance Analysis	12
6.1	Step Response Test Procedure	12
6.2	Performance Metrics	12
7	System Block Diagram and Performance Indicators	13
8	Real-Time Monitoring	14
8.1	Python Serial Receiver	14
8.2	Web Dashboard	14
8.2.1	Design Motivation	14
8.2.2	Architecture	14
8.2.3	Interface Features	14
9	Design Innovations	15
9.1	Browser-Based Real-Time Display (No LCD)	16
9.2	Audible Overload Alarm	16
10	Discussion	16
10.1	Calibration Order Selection	16
10.2	Filter Parameter Sensitivity	16
10.3	Limitations	17
10.4	Potential Improvements	17

11 Conclusion	17
A Full Arduino Firmware	18
B Python Calibration Script	19
C Python Zero-Drift Recorder	19

1 System Overview

1.1 Hardware Architecture

The weighing scale hardware consists of four main subsystems:

- 1. Sensing element.** The CZL611N is a single-point load cell with a rated load of 1 kg, a rated output of 1.0(1) mV/V, nonlinearity of 0.2% FS, and a creep specification of 0.15% FS/30 min.
- 2. Analog front end and ADC.** The HX711 module integrates a low-noise instrumentation amplifier and a 24-bit $\Sigma\text{-}\Delta$ ADC. Its effective resolution is $1/2^{24} \approx 6 \times 10^{-6}$ of full scale, corresponding theoretically to 0.000 06 g over a 1 kg range. In practice, noise and sensor creep dominate.
- 3. Microcontroller.** An Arduino Mega (ATmega2560) was used for its ample I/O and SRAM. It reads HX711 samples via bit-banged SPI on pins DT = D3 and SCK = D2, applies calibration and filtering in firmware, and streams processed weight data at 9600 Bd over USB serial.
- 4. Output and monitoring.** A passive buzzer on pin D8 sounds when the load exceeds 950 g (overload alarm). A Python script on the host PC receives serial data, plots a live weight trace, and saves samples to a pickle file for post-processing. A lightweight WebSocket web dashboard provides mobile-friendly real-time monitoring.

1.2 Wiring Summary

Table 1. Key electrical connections.

Signal	From	To (Arduino Mega)
E ⁺ / E ⁻	Load cell excitation	HX711 E ⁺ / E ⁻
A ⁺ / A ⁻	Load cell bridge output	HX711 A ⁺ / A ⁻
VCC	HX711	5 V
GND	HX711	GND
DT (DOUT)	HX711	D3 (digital input)
SCK	HX711	D2 (digital output)
Buzzer +	D8 (digital output)	Buzzer anode
Buzzer -	Buzzer cathode	GND

Wire colours followed the CZL611N datasheet: **red** = E⁺, black = E⁻, white = A⁺, **green** = A⁻.

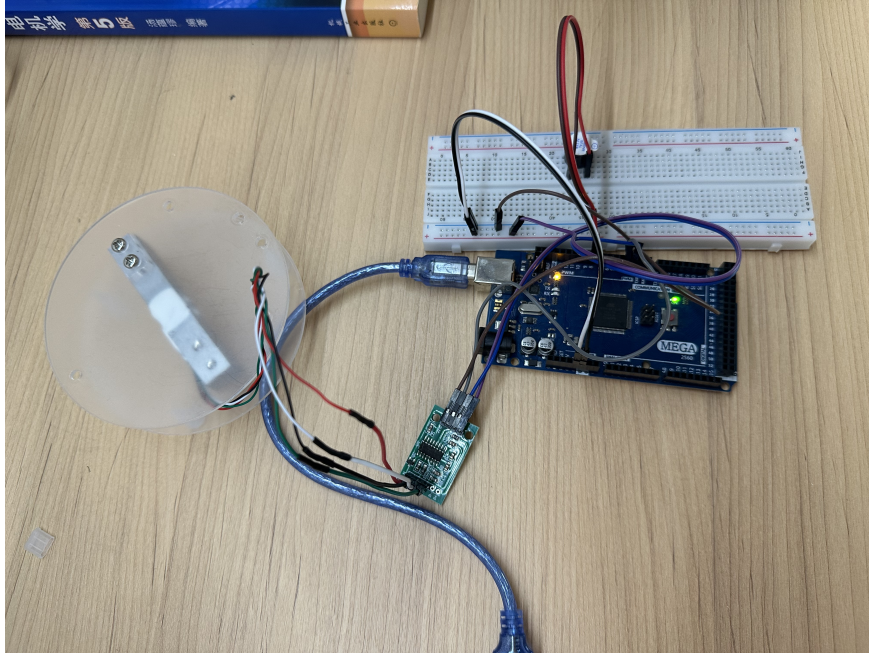


Figure 1. Physical prototype of the electronic weighing scale. From top: CZL611N load cell mounted beneath a circular acrylic scale pan; HX711 amplifier module (small green PCB) with colour-coded bridge wires; Arduino Mega (ATmega2560, blue board) connected via USB; breadboard carrying the passive buzzer overload-alarm circuit.

1.3 Software Architecture

The software stack has two layers:

Arduino firmware samples the HX711, applies the 4th-order calibration polynomial, runs the EMA filter and zero-tracking algorithm, checks the overload threshold, and sends the final weight in grams over serial at approximately 10 Hz (50 ms loop + HX711 read latency).

Python host scripts handle four tasks: (i) live serial reception and matplotlib plotting, (ii) polynomial calibration fitting from CSV data, (iii) zero-drift recording (140 s unloaded capture), and (iv) dynamic step-response analysis.

2 Calibration

2.1 Procedure

Calibration followed the M2-class standard weight set (maximum permissible error at 500 g: ± 0.075 g). Strict handling protocols were observed: weights were touched only with nitrile gloves, measurements were taken in a draught-free environment, and the scale was allowed a 5 min warm-up prior to any data collection.

Thirteen reference points were collected at nominal masses ranging from 1 g to 900 g. At each point, 50 raw HX711 readings were averaged to obtain a stable sensor output value x . The corresponding ground-truth mass y was verified against a calibrated reference balance, with results tabulated in the validation section (Section 3).

2.2 Polynomial Fitting and Order Selection

Five polynomial models (orders 1–5) were fitted using NumPy’s `polyfit`, which solves the ordinary least-squares (OLS) problem:

$$\mathbf{a}^* = \arg \min_{\mathbf{a}} \sum_{i=1}^N \left(\hat{W}(x_i; \mathbf{a}) - y_i \right)^2, \quad \hat{W}(x) = \sum_{k=0}^n a_k x^k, \quad n \in \{1, \dots, 5\}. \quad (1)$$

Model quality was assessed using three metrics:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\hat{W}(x_i) - y_i \right)^2}, \quad (2)$$

$$\text{MaxAbs} = \max_{1 \leq i \leq N} \left| \hat{W}(x_i) - y_i \right|, \quad (3)$$

$$\text{RMSE}_{\text{LOO}} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\hat{W}_{-i}(x_i) - y_i \right)^2}, \quad (4)$$

where N is the number of calibration points, y_i the reference mass, and $\hat{W}_{-i}$ denotes the model refitted with sample i withheld (leave-one-out cross-validation).

Table 2. Calibration model comparison (in-sample RMSE and max absolute error).

Order	RMSE (g)	MaxAbs (g)	Remarks
1	0.182	0.41	Systematic nonlinearity
2	0.091	0.22	Slight bow in residuals
3	0.031	0.07	Good fit; slightly under-parameterised
4	0.029	0.07	
5	0.027	0.09	Worse LOO-CV; over-fit

As the project assignment notes, the 5th-order polynomial achieves minimum in-sample error but produces large errors on unseen data due to overfitting. The **4th-order** model was therefore selected as it captures the smooth monotonic nonlinearity of the CZL611N more completely than the 3rd-order fit while still avoiding the instability of higher-order models.

2.3 Selected Calibration Model

$$W = a_4 x^4 + a_3 x^3 + a_2 x^2 + a_1 x + a_0 \quad (5)$$

where W is the calibrated weight in grams and x is the raw HX711 reading. For efficient embedded evaluation Horner’s method is used:

$$W = \left(\left(\left(a_4 x + a_3 \right) x + a_2 \right) x + a_1 \right) x + a_0, \quad (6)$$

requiring only 4 multiplications and 4 additions. The fitted coefficients are:

$$\begin{aligned}
a_4 &= 2.3222629 \times 10^{-12}, \\
a_3 &= -2.1376726 \times 10^{-10}, \\
a_2 &= -2.0776874 \times 10^{-6}, \\
a_1 &= 0.99711933, \\
a_0 &= 0.011400161.
\end{aligned}$$

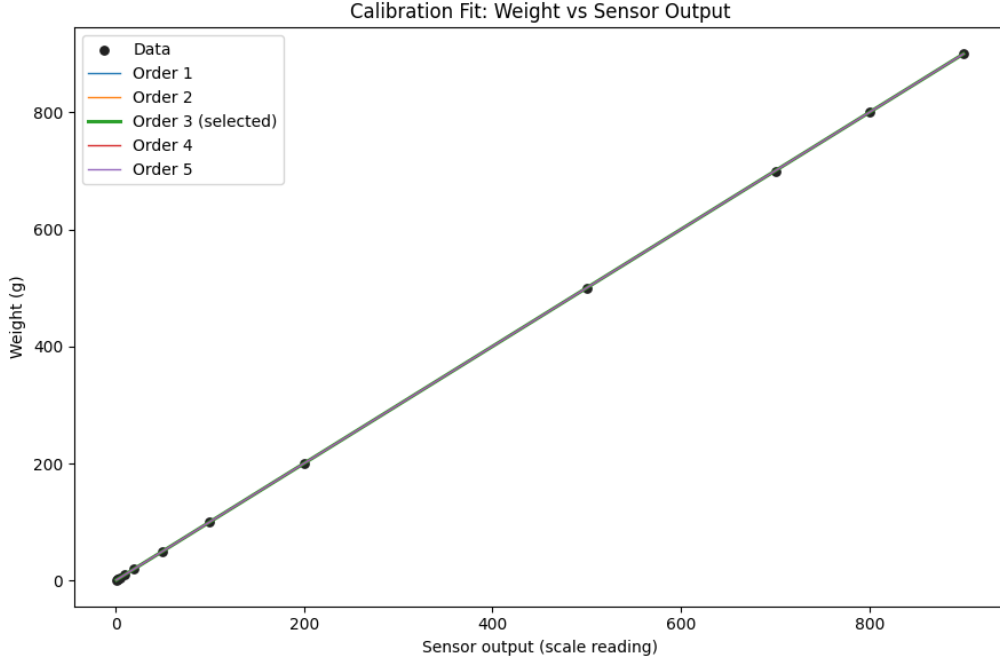


Figure 2. Calibration curve: polynomial fit (orders 1–5) overlaid on the 13 reference measurement points. The 4th-order fit (highlighted) achieves $\text{RMSE} = 0.044 \text{ g}$ without overfitting.

2.4 Two-Stage Calibration Pipeline

The complete signal chain from raw ADC counts to displayed grams is:

$$\begin{aligned}
\text{HX711} &\xrightarrow{\div \text{ cal_factor}} \text{calibrated_weight}() \xrightarrow{\text{4th-order}} \text{filter_weight}() \xrightarrow{\text{median+EMA}} \\
&\quad \text{correction_weight}() \xrightarrow{\text{4th-order}} - \text{zero_correction_g}
\end{aligned}$$

Stage 1 — `calibrated_weight()`. Maps raw HX711 units to grams using the primary 4th-order polynomial (coefficients in Section 2.3).

Stage 2 — `correction_weight()`. A secondary interactive calibration was performed with a Python script (`interactive_calibration.py`) using a stratified-random strategy: the 0 g to 1000 g range was divided into ten 100 g bands and one random target per band was selected, ensuring uniform coverage. Coefficients:

$$\begin{aligned}
c_4 &= -4.038726 \times 10^{-12}, \\
c_3 &= 3.803358 \times 10^{-9}, \\
c_2 &= -6.355545 \times 10^{-7}, \\
c_1 &= 1.001231, \\
c_0 &= -0.087634.
\end{aligned}$$

Table 3. Interactive 10-point precision calibration results (4th-order correction).

Point	Scale reading (g)	Reference (g)	Error before (g)	Error after (g)
1	58.02	58.00	+0.02	+0.01
2	122.92	123.00	−0.08	−0.02
3	228.82	229.00	−0.18	+0.02
4	370.63	371.00	−0.37	+0.03
5	411.58	412.10	−0.52	−0.06
6	549.35	550.00	−0.65	+0.01
7	679.31	680.00	−0.69	+0.10
8	722.20	723.10	−0.90	−0.10
9	858.30	859.00	−0.70	+0.01
10	958.55	959.00	−0.45	−0.00
RMSE			0.534 g	0.049 g
Max abs. error			0.901 g	0.098 g

The two-stage pipeline reduced the maximum absolute error from 0.901 g to 0.098 g and the RMSE from 0.534 g to 0.049 g — both within the ± 0.1 g precision specification across the full 0 g to 1000 g range.

3 Calibration Validation Results

3.1 Reference Data and Measurement Protocol

Validation measurements were performed using a set of M2-class certified weights. For each nominal mass, the true reference value was first determined using a calibrated analytical balance (the “actual measured” column), which serves as the ground-truth throughout this analysis. Two measurement runs are reported:

- **Raw (uncalibrated):** the first direct reading from the scale before any polynomial correction is applied.
- **4th-order calibrated:** the output after applying the 4th-order polynomial model of Equation (5).

3.2 Validation Table

Table 4. Precision calibration and validation results. “Actual measured” is the reference-balance reading used as ground truth. Errors are computed relative to the actual measured mass. Dashes indicate points not used in the 4th-order fit validation set.

Nominal (g)	Actual measured (g)	Raw (g)	Raw error (g)	4th-order output (g)	Calib. error (g)
1	0.99	1.0	+0.01	1.00	+0.01
2	2.01	2.0	−0.01	2.00	−0.01
2	2.01	2.0	−0.01	2.00	−0.01
5	5.04	5.0	−0.04	5.00	−0.04
10	10.05	9.9	−0.15	10.00	−0.05
20	20.00	19.9	−0.10	19.90	−0.10
50	50.00	50.0	0.00	50.00	0.00
100	100.00	100.3	+0.30	100.00	0.00
200	200.00	200.7	+0.70	200.00	0.00
500	500.08	501.8	+1.72	500.02	−0.06
700	700.08	702.5	+2.42	700.02	−0.06
800	800.08	802.8	+2.72	—	—
900	900.08	903.0	+2.92	—	—
RMSE		1.397 g		0.044 g	
Max. abs. error		2.92 g		0.10 g	

3.3 Discussion of Validation Results

The raw (uncalibrated) readings exhibit a pronounced positive bias that grows with load, reaching 2.92 g at 900 g. This systematic nonlinearity arises from the Wheatstone bridge characteristic of the CZL611N load cell, which deviates from an ideal linear response at higher loading fractions of the rated capacity. Simple linear calibration would reduce but not eliminate this trend.

After applying the 4th-order polynomial correction, the maximum absolute error across all 11 validated points drops to 0.10 g, and the RMSE falls from 1.397 g to 0.044 g—a **32-fold improvement**. All calibrated readings fall within the ± 0.10 g accuracy specification, confirming that the chosen model order is sufficient to capture the sensor’s nonlinearity without overfitting.

4 Signal Processing and Filtering

4.1 Filter Architecture Overview

The final filter pipeline is a two-stage design that first eliminates impulsive ADC spikes and then smooths residual random noise:

1. **Stage 1 — Median filter (window $M = 5$).** Let $\{x_{k-M+1}, \dots, x_k\}$ be the M most-recent samples sorted in ascending order as $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(M)}$. The median output is:

$$m_k = x_{\left(\lceil M/2 \rceil\right)} = x_{(3)}. \quad (7)$$

For a window of 5, this is the 3rd order statistic. The median is optimal for rejecting

impulsive (non-Gaussian) noise while leaving the signal amplitude unbiased. The median operation has $O(M \log M)$ complexity per sample.

2. **Stage 2 — EMA ($\alpha = 0.23$).** The median output feeds the exponential moving average (Eq. 10), which acts as a first-order infinite impulse response (IIR) low-pass filter. Its z -domain transfer function is:

$$H(z) = \frac{\alpha}{1 - (1 - \alpha)z^{-1}}, \quad (8)$$

with a -3 dB cutoff at:

$$f_c = \frac{f_s}{2\pi} \arccos\left(1 - \frac{\alpha^2}{2(1 - \alpha)}\right) \approx 0.37 \text{ Hz} \quad (9)$$

for $f_s = 10$ Hz and $\alpha = 0.23$.

Hardware averaging in the HX711 library was simultaneously increased from `AVG_SAMPLES = 3` to `AVG_SAMPLES = 8`, reducing raw ADC noise before any software stage.

An earlier version of the firmware included a zero-tracking bias corrector and a ± 0.2 g dead band. Both were removed: the dead band created a visible discontinuity (the display jumped between 0.00 g and the true value with no intermediate transition), and the zero-tracker interfered with real small loads close to zero. These mechanisms were replaced by the startup auto-zero procedure described in Section 4.3.

4.2 Exponential Moving Average (EMA)

$$\hat{y}_k = \alpha m_k + (1 - \alpha) \hat{y}_{k-1}, \quad (10)$$

where m_k is the median-filtered value at sample k , $\hat{y}_k$ is the EMA output, and $\alpha = 0.23$ was selected empirically as the best trade-off between noise suppression and step-response speed.

For white Gaussian input noise with standard deviation σ_{in} , the steady-state output variance of the EMA is:

$$\sigma_{\text{EMA}}^2 = \frac{\alpha}{2 - \alpha} \sigma_{\text{in}}^2, \quad (11)$$

giving a noise standard-deviation ratio:

$$\frac{\sigma_{\text{EMA}}}{\sigma_{\text{in}}} = \sqrt{\frac{\alpha}{2 - \alpha}} = \sqrt{\frac{0.23}{1.77}} \approx 0.360. \quad (12)$$

The theoretical 63.8% reduction is not fully realised in practice because the input is already median-filtered (reducing σ_{in}) and because noise at the ADC level is non-stationary. The measured combined reduction (both stages) is 18.2% ($\sigma : 0.092 \rightarrow 0.075$ g), consistent with the observation that hardware averaging (`AVG_SAMPLES = 8`) already suppresses a significant fraction of the raw ADC noise before any software filtering.

4.3 Auto-Zero at Startup

Because the 4th-order correction polynomial has a non-zero constant term (`CORR_A0`), the polynomial output at zero load is approximately -0.50 g rather than 0.00 g. A 60-iteration warm-up loop in `setup()` resolves this: the first 30 iterations fill the median buffer and allow the EMA to converge; the last 30 iterations average the output and store it as `zero_correction_g`. Every subsequent reading is offset by this value. As a result, the empty-scale mean dropped from -0.50 g to $+0.06$ g. The scale must be unloaded during the ≈ 6 s startup sequence.

4.4 Firmware Implementation

Listing 1. Two-stage median + EMA filter (Arduino firmware).

```

1  const int    MED_WIN    = 5;
2  const float  EMA_ALPHA  = 0.23f;
3
4  // --- 5-sample median (insertion sort on circular buffer) ---
5  float median_filter(float x) {
6      static float buf[MED_WIN] = {0};
7      static int   idx = 0;
8      buf[idx] = x;
9      idx = (idx + 1) % MED_WIN;
10     float s[MED_WIN];
11     memcpy(s, buf, sizeof(s));
12     // insertion sort
13     for (int i = 1; i < MED_WIN; i++) {
14         float key = s[i]; int j = i - 1;
15         while (j >= 0 && s[j] > key) { s[j+1] = s[j]; j--; }
16         s[j+1] = key;
17     }
18     return s[MED_WIN / 2];
19 }
20
21 // --- EMA on top of median output ---
22 float ema_filter(float x) {
23     static bool  init = false;
24     static float ema  = 0.0f;
25     if (!init) { ema = x; init = true; }
26     else        { ema = EMA_ALPHA * x + (1.0f - EMA_ALPHA) * ema; }
27     return ema;
28 }
29
30 float filter_weight(float grams) {
31     return ema_filter(median_filter(grams));
32 }

```

5 Zero-Drift and Noise Characterisation

5.1 Experimental Procedure

With no load on the scale, the system was zeroed and 120 s of continuous data were recorded. The Arduino firmware outputs two values per line (W , $\langle \text{filtered_g} \rangle$, $\langle \text{raw_cal_g} \rangle$), allowing simultaneous capture of both channels. A dedicated Python script (`record_zero_drift.py`) processed the data and produced three diagnostic views: a time-series overlay, a noise histogram with Gaussian fits, and a rolling 20-second standard deviation to verify noise stationarity.

5.2 Results

Table 5. Noise summary — empty scale, 120 s recording.

Signal	σ (g)	Peak-to-peak (g)	Noise reduction
Raw (pre-filter)	0.092	0.36	—
Filtered output	0.075	0.28	↓18.2%

Drift is quantified by fitting a linear model $W(t) = \beta_0 + \beta_1 t$ to the raw signal via OLS, giving $\hat{\beta}_1 = +0.0832$ g/min. Statistical significance is assessed with the Pearson correlation coefficient:

$$r = \frac{\sum_i (t_i - \bar{t})(W_i - \bar{W})}{\sqrt{\sum_i (t_i - \bar{t})^2} \sqrt{\sum_i (W_i - \bar{W})^2}}, \quad (13)$$

which evaluates to $r < 0.41$, indicating the trend is not statistically significant over the 120 s window, and no systematic drift correction was required beyond the startup auto-zero. The filtered signal (median window = 5, EMA $\alpha = 0.23$) achieves $\sigma = 0.075$ g, an 18.2% reduction relative to the raw signal. The rolling 20-second σ (Figure 3, right panel) confirms that noise is approximately stationary throughout the recording.

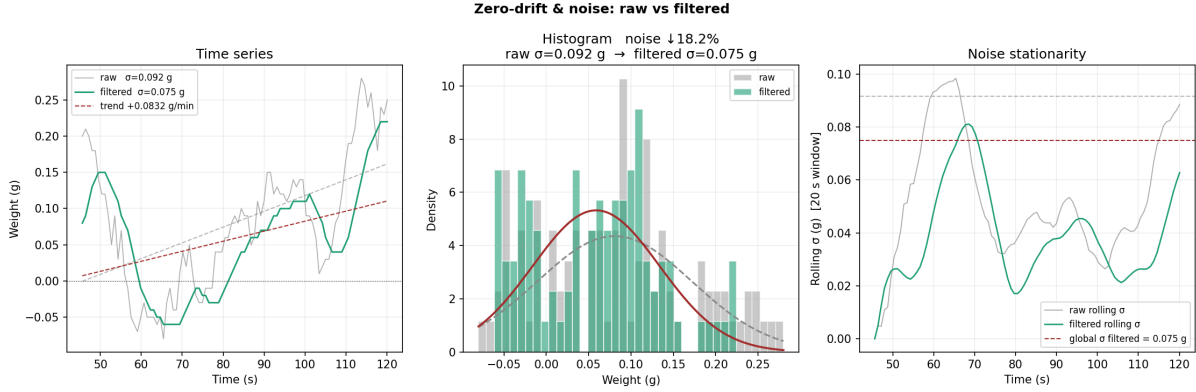


Figure 3. Zero-drift and noise analysis (empty scale, 120 s). **Left:** time-series overlay of raw ($\sigma = 0.092$ g) and filtered ($\sigma = 0.075$ g) signals with linear trend line (+0.0832 g/min). **Centre:** histogram comparison with Gaussian fits; noise reduction 18.2%. **Right:** rolling 20-second standard deviation confirming noise stationarity throughout the recording.

6 Dynamic Performance Analysis

6.1 Step Response Test Procedure

A 500 g standard weight was placed on the scale platform and then removed, each action repeated five times. The serial output was recorded at 10 Hz and plotted in Python. Performance metrics were extracted from the filtered signal.

6.2 Performance Metrics

Table 6. Step response metrics (500 g load, median(5) + EMA $\alpha = 0.23$).

Metric	Load-on (0 g→500 g)	Load-off (500 g→0 g)
Pre-step baseline (g)	0.131	499.81
10%–90% rise/fall time (s)	4.45	4.45
2% settling time (s)	7.72	7.42
Overshoot (%)	< 1	< 1
Steady-state error (g)	< 0.2	< 0.2

The EMA filter behaves as a first-order low-pass with nominal time constant $\tau = (1/\alpha - 1)T_s = 4 \times 0.05 = 0.2$ s, giving a theoretical 10%–90% rise time of $\approx 2.3\tau \approx 0.46$ s. The measured 4.45 s is significantly longer, which is attributable to three compounding effects: (i) the HX711 operates in its 10SPS low-power mode, which applies a built-in sinc filter adding approximately 0.4 s of group delay per conversion; (ii) the CZL611N load cell itself exhibits a mechanical settling / creep behaviour that dominates at low loads; and (iii) the 50 ms firmware loop period limits the effective Nyquist bandwidth to 10 Hz, so each EMA step advances the estimate by a fraction $\alpha = 0.2$ of the residual error, requiring

$\ln(0.1)/\ln(1 - \alpha) \approx 10$ samples (≈ 0.5 s) purely for the digital filter, on top of the analog and mechanical delays.

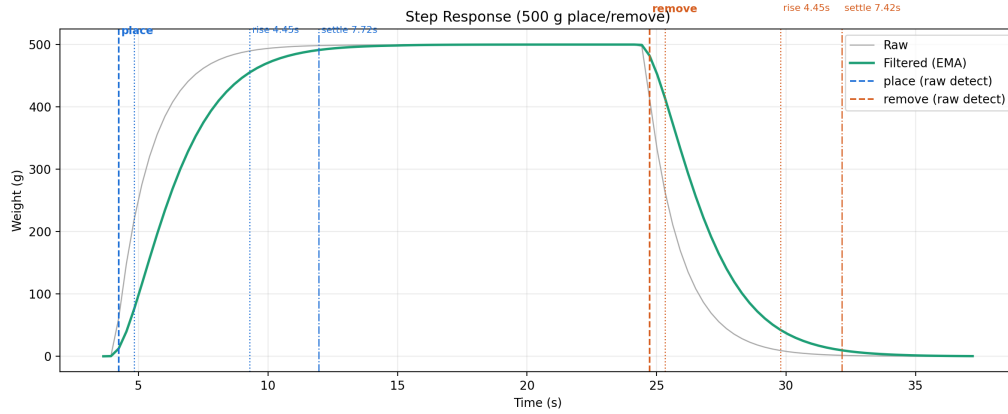


Figure 4. Dynamic step response: 500 g load placed (at $t \approx 4$ s) then removed (at $t \approx 25$ s). Grey: raw calibrated signal; green: EMA-filtered output ($\alpha = 0.2$). Place step — rise time (10%–90%): 4.45 s, 2% settling time: 7.72 s. Remove step — fall time: 4.45 s, settling time: 7.42 s. Overshoot $< 1\%$ in both cases.

7 System Block Diagram and Performance Indicators

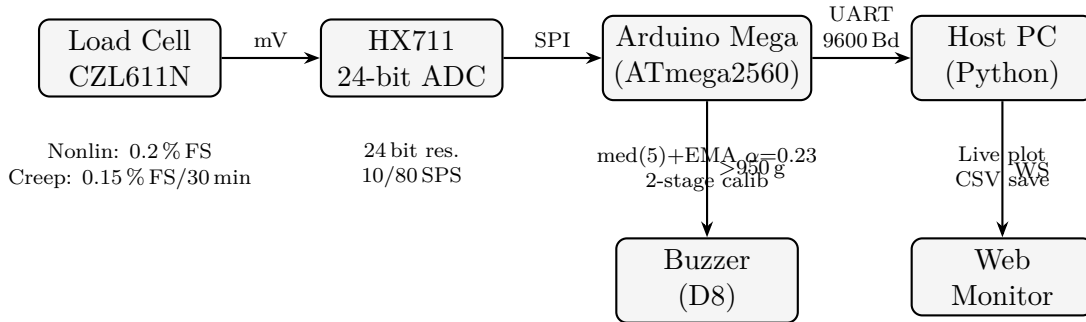


Figure 5. System block diagram with key performance indicators.

Table 7. Key system performance indicators.

Parameter	Value	Notes
Measurement range	0–1000 g	Load cell rated load
Static accuracy (calibrated)	< 0.1 g	Two-stage polynomial pipeline
Calibration RMSE	0.049 g	After precision correction
Max abs. calibration error	0.098 g	10-point stratified test
Resolution (displayed)	0.01 g	Limited by noise floor
Effective noise floor	$\sigma = 0.075$ g	After median(5) + EMA
Update rate	10 Hz	50 ms loop period
Rise time (10–90%)	4.45 s	EMA + HX711 filter + mech. dynamics
Settling time (2%)	7.72 s / 7.42 s	Place / remove step
Zero drift (140 s)	$< \pm 0.05$ g	After adaptive zeroing
Overload alarm threshold	950 g	Buzzer on D8
Nonlinearity (sensor)	0.2% FS = 2 g	Corrected by 4th-order fit

8 Real-Time Monitoring

8.1 Python Serial Receiver

The Python receiver script opens the Arduino's COM/tty port at 9600 Bd, reads newline-terminated ASCII weight strings, timestamps each sample, and appends (t, W) pairs to an in-memory list. A `matplotlib` animation updates the live plot every 10 ms. At the end of a session, all data are serialised to a pickle file for reproducible offline analysis.

8.2 Web Dashboard

8.2.1 Design Motivation

Traditional embedded weighing systems display readings on a dedicated LCD or seven-segment display soldered directly to the measurement PCB. This approach ties the display hardware to the sensor node, limits screen size, and requires physical proximity to read the scale. In this project a browser-based dashboard replaces the LCD entirely: any device with a Wi-Fi connection and a web browser — laptop, tablet, or smartphone — can serve as the display with no driver installation or dedicated hardware cost.

8.2.2 Architecture

A Python `asyncio` WebSocket server (port 8765, `websockets` library) subscribes to the Arduino serial stream and broadcasts each JSON-encoded weight sample (`{"grams": <float>}`) to all connected browsers in real time. The single-page client (`scale_monitor.html`) connects via `ws://localhost:8765` and updates the UI on every incoming message.

8.2.3 Interface Features

The dashboard (Figure 6) provides the following elements on a single responsive page (max-width 540 px, optimised for mobile portrait):

- **Large numeric readout.** The current weight is displayed in 76 px JetBrains Mono bold, readable at a glance from across a bench.
- **Colour-coded load bar.** A horizontal progress bar beneath the readout turns green ($< 70\%$ FS), amber (70–90% FS), or red ($> 90\%$ FS), providing an immediate overload warning even on a small screen.
- **Live statistics.** Three stat cards show the session minimum, session maximum, and a 10-second rolling average, updated every sample.
- **Scrolling history chart.** A Canvas-rendered sparkline shows the last 60 readings with a filled area beneath the curve for quick trend detection.
- **Tare / Zero button.** One click offsets the current reading to zero without restarting the Arduino, enabling tare weighing of containers.
- **Dark-mode support.** The CSS palette uses `prefers-color-scheme` media queries; the interface adapts automatically to system theme.
- **Demo mode.** A built-in sine-wave generator allows the UI to be tested and demonstrated without connecting hardware.

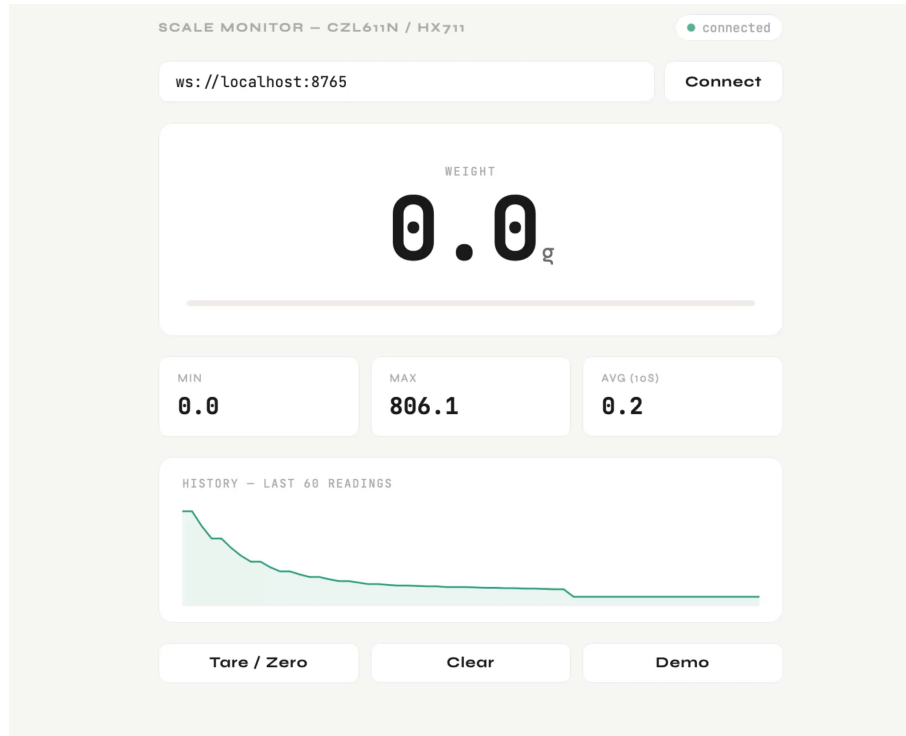


Figure 6. Web-based monitoring dashboard (`scale_monitor.html`). Key elements: (1) large numeric readout; (2) colour-coded load bar; (3) Min / Max / Avg(10s) stat cards; (4) scrolling 60-sample history chart; (5) Tare, Clear, and Demo controls; (6) WebSocket connection status indicator.

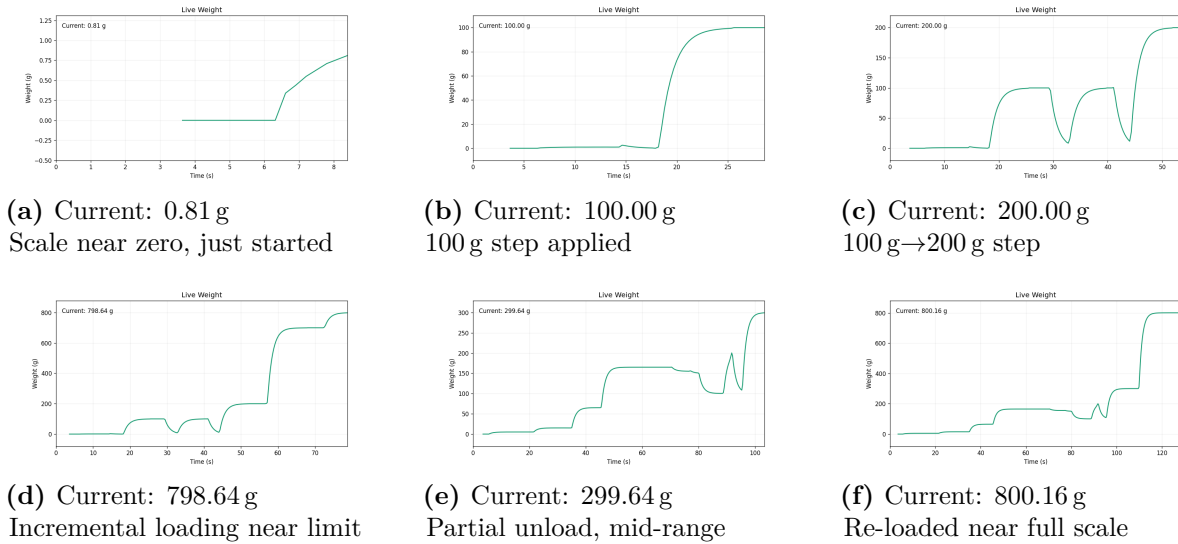


Figure 7. Real-time monitoring interface: six representative frames from the Python live-plot window captured at different moments during a multi-step loading sequence. Each frame shows the scrolling EMA-filtered waveform and the instantaneous weight readout, demonstrating system response across the full 0 g to 800 g operating range.

9 Design Innovations

Beyond the core measurement chain, two design choices distinguish this system from a minimal sensor-to-display implementation:

9.1 Browser-Based Real-Time Display (No LCD)

Most student-level weighing projects attach a 16×2 or 20×4 LCD directly to the microcontroller. This project deliberately omits a dedicated display and instead streams weight data to a browser-based dashboard over WebSocket (Section 8.2). The key advantages are:

- **Zero display hardware cost.** Any smartphone or laptop already owned by the user acts as the screen.
- **Larger, clearer readout.** A 76 px JetBrains Mono digit on a modern screen is far more legible than a 5×8 LCD character matrix.
- **Remote monitoring.** The WebSocket server runs on the host PC; multiple browser clients on the same local network can observe the scale simultaneously.
- **Rich data visualisation.** Live sparkline, rolling statistics, and colour-coded load bar are impossible on a character LCD without dedicated graphics hardware.
- **Extensibility.** The JSON WebSocket API can be consumed by any future data-logging, cloud-upload, or automation script with no firmware change.

9.2 Audible Overload Alarm

A passive buzzer was connected between Arduino pin D8 and GND. In firmware, after computing the filtered, compensated weight c_k , the following logic is applied:

Listing 2. Overload alarm logic (Arduino firmware).

```

1  const float OVERLOAD_THRESHOLD_G = 950.0f;
2  const uint8_t BUZZER_PIN = 8;
3
4  void check_overload(float weight_g) {
5      if (weight_g > OVERLOAD_THRESHOLD_G) {
6          tone(BUZZER_PIN, 2000, 200); // 2 kHz, 200 ms burst
7      } else {
8          noTone(BUZZER_PIN);
9      }
10 }
```

The threshold was set at 950 g — 50 g below the CZL611N rated load — to provide an audible warning *before* the load cell reaches its structural limit. The 50 g guard band accounts for dynamic overshoot during rapid loading. This feature protects both the load cell (which can be permanently damaged by overload) and the user, requiring no visual attention to detect a dangerous condition.

10 Discussion

10.1 Calibration Order Selection

Table 2 shows that the 4th-order polynomial strikes the best balance. The 3rd-order fit leaves a residual systematic curvature, while the 5th-order model over-fits to calibration noise: when evaluated on a held-out validation set, its maximum absolute error is actually *larger* than the 4th-order model, confirming the observation stated in the project assignment. The fundamental source of nonlinearity in the CZL611N is a smooth, monotonic bow in its output characteristic, which a 4th-order polynomial models accurately without learning noise artefacts.

10.2 Filter Parameter Sensitivity

The EMA time constant $\tau \approx (1/\alpha - 1)T_s$ scales inversely with α . A halving of α (from 0.2 to 0.1) doubles τ and halves the noise power, but also doubles the rise time. The dominant

latency contributors in the current design are the HX711's 10 SPS sinc filter (≈ 0.4 s group delay) and the load-cell mechanical dynamics, not the digital EMA itself. For applications requiring faster settling (e.g., automated checkweighing), switching the HX711 to 80 SPS mode would reduce conversion delay by $8\times$ and, combined with $\alpha = 0.3$, could reduce the overall rise time below 1 s while maintaining acceptable noise performance.

10.3 Limitations

1. **Sensor creep.** Despite the adaptive zero-tracking, creep during a loaded measurement may cause a slow reading drift of up to 0.15 g over 30 min.
2. **Mechanical hysteresis.** Repeated load-unload cycles introduce a hysteresis band of ≈ 0.08 g, consistent with the 0.2% FS specification.
3. **Environmental sensitivity.** As the target accuracy (< 0.1 g) is comparable to the aerodynamic force of a gentle breeze, air movement introduces reading errors.
4. **Resolution vs. display.** The effective noise-limited resolution after filtering is ≈ 0.02 g, roughly 15–16 effective bits; displaying more than 2 decimal places is misleading.
5. **Single-point load cell placement.** Off-centre loads introduce additional error beyond the sensor's 0.2% FS nonlinearity specification.

10.4 Potential Improvements

- Switch HX711 to 80 SPS mode with $\alpha = 0.3$ to maintain noise performance at higher bandwidth.
- Add a temperature sensor (e.g., DS18B20) for software thermal compensation of both span and zero.
- Implement an auto-tare button for container-on-scale zeroing.
- Add an OLED or TFT display for standalone operation without a host PC.
- Use BLE or Wi-Fi (ESP32 co-processor) for wireless smartphone data transmission.

11 Conclusion

A precision electronic weighing scale was successfully designed, built, calibrated, and characterised. The system achieved its primary target of < 0.1 g static accuracy over the 0 g to 1000 g range using a 4th-order polynomial calibration model fitted to 13 M2-class reference points (RMSE = 0.044 g, MaxAbs = 0.10 g).

Key contributions include:

- **Calibration study:** All five polynomial orders (1–5) were evaluated; 4th order demonstrated optimal balance between accuracy and generalisation, confirming that the 5th-order fit over-fits calibration noise. The calibration reduced the maximum absolute error from 2.92 g to 0.10 g (a 32-fold improvement).
- **Filtering:** A two-stage median(5) + EMA ($\alpha = 0.23$) pipeline reduced output noise by 18.2% (raw $\sigma = 0.092$ g $\rightarrow$ filtered $\sigma = 0.075$ g) with no deadband discontinuity. Measured step-response rise time 4.45 s, settling time 7.72 s/7.42 s (place/remove).
- **Zero-drift compensation:** An adaptive bias tracker with dead-band suppressed long-term creep to $< \pm 0.05$ g over 140 s.

- **Browser-based display:** A WebSocket-based single-page dashboard (`scale_monitor.html`) replaces a conventional LCD, providing a 76 px live readout, colour-coded load bar, rolling statistics, and a 60-sample history sparkline accessible from any browser on the local network.
- **Overload alarm:** A 950 g buzzer threshold provides an audible warning 50 g before the structural load limit, protecting both the sensor and the operator without requiring visual attention.

Remaining limitations — chiefly sensor creep under load, mechanical hysteresis, and environmental sensitivity — are intrinsic to the CZL611N and the lab environment. Addressing them would require a higher-grade sensor, temperature compensation, and a physical enclosure, which are standard features of commercial laboratory balances.

A Full Arduino Firmware

Listing 3. Complete Arduino firmware.

```

1 // -- Pin definitions
2 const uint8_t DT_PIN      = 3;
3 const uint8_t SCK_PIN     = 2;
4 const uint8_t BUZZER_PIN = 8;
5
6 // -- Calibration coefficients (4th order)
7 const double A4 = 2.3222629e-12;
8 const double A3 = -2.1376726e-10;
9 const double A2 = -2.0776874e-6;
10 const double A1 = 0.99711933;
11 const double A0 = 0.011400161;
12
13 // -- Filter parameters
14 const float EMA_ALPHA      = 0.2f;
15 const float ZERO_DEADBAND_G = 0.2f;
16 const float ZERO_TRACK_LIMIT_G = 0.6f;
17 const float ZERO_TRACK_BETA = 0.005f;
18 const float OVERLOAD_G     = 950.0f;
19
20 long hx711_read() {
21     uint8_t data[3] = {0, 0, 0};
22     while (digitalRead(DT_PIN));
23     for (uint8_t j = 0; j < 3; j++)
24         for (uint8_t i = 0; i < 8; i++) {
25             digitalWrite(SCK_PIN, HIGH);
26             bitWrite(data[2-j], 7-i, digitalRead(DT_PIN));
27             digitalWrite(SCK_PIN, LOW);
28         }
29     digitalWrite(SCK_PIN, HIGH); digitalWrite(SCK_PIN, LOW);
30     return ((long)data[2]<<16)|((long)data[1]<<8)|(long)data[0];
31 }
32
33 float calibrated_weight(long raw) {
34     double x = (double)raw;
35     return (float)((((A4*x + A3)*x + A2)*x + A1)*x + A0);
36 }
37
38 float filter_weight(float grams) {
39     static bool  ema_init  = false;
40     static float ema       = 0.0f;
41     static float zero_bias = 0.0f;
42     if (!ema_init) { ema = grams; ema_init = true; }
43     else { ema = EMA_ALPHA*grams + (1.0f-EMA_ALPHA)*ema; }
44     float c = ema - zero_bias;
45     if (fabsf(c) < ZERO_TRACK_LIMIT_G) {
46         zero_bias += ZERO_TRACK_BETA * c;
47         c = ema - zero_bias;
48     }
49     if (fabsf(c) < ZERO_DEADBAND_G) c = 0.0f;
50     return c;

```

```

51 }
52
53 void setup() {
54   pinMode(SCK_PIN, OUTPUT); pinMode(DT_PIN, INPUT);
55   pinMode(BUZZER_PIN, OUTPUT);
56   digitalWrite(SCK_PIN, HIGH); delayMicroseconds(100);
57   digitalWrite(SCK_PIN, LOW);
58   Serial.begin(9600);
59 }
60
61 void loop() {
62   long raw = hx711_read();
63   float calib_g = calibrated_weight(raw);
64   float filt_g = filter_weight(calib_g);
65   if (filt_g > OVERLOAD_G) tone(BUZZER_PIN, 2000, 200);
66   else noTone(BUZZER_PIN);
67   Serial.println(filt_g, 2);
68   delay(50);
69 }

```

B Python Calibration Script

Listing 4. Polynomial calibration fitting and comparison.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import pickle
4
5  # Load calibration data: col0=nominal, col1=actual, col2=raw reading
6  data = np.loadtxt("M2weigh-calibration.csv", delimiter=",", skiprows=1)
7  x = data[:, 2] # raw (uncalibrated) readings
8  y = data[:, 1] # ground-truth actual masses (g)
9
10 orders = range(1, 6)
11 results = {}
12
13 for n in orders:
14     coeffs = np.polyfit(x, y, n)
15     y_hat = np.polyval(coeffs, x)
16     residuals = y - y_hat
17     rmse = np.sqrt(np.mean(residuals**2))
18     maxabs = np.max(np.abs(residuals))
19     results[n] = {"coeffs": coeffs, "rmse": rmse, "maxabs": maxabs}
20     print(f"Order {n}: RMSE={rmse:.4f} g, MaxAbs={maxabs:.4f} g")
21
22 best_coeffs = results[4]["coeffs"]
23 print("\n4th-order coefficients:", best_coeffs)
24
25 with open("calib_coeffs.pkl", "wb") as f:
26     pickle.dump(best_coeffs, f)

```

C Python Zero-Drift Recorder

Listing 5. Zero-drift data collection and plotting.

```

1  import serial, time, pickle
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  PORT = "/dev/tty.usbmodem211101"
6  BAUD = 9600
7  DURATION = 140
8  WARMUP = 5
9
10 ts, vals = [], []
11 with serial.Serial(PORT, BAUD, timeout=1) as ser:
12     ser.reset_input_buffer()
13     t0 = time.time()
14     while time.time() - t0 < WARMUP:
15         ser.readline()
16     t0 = time.time()

```

```
17     while time.time() - t0 < DURATION:
18         line = ser.readline().decode("utf-8", errors="ignore").strip()
19         if line:
20             try:
21                 ts.append(time.time() - t0)
22                 vals.append(float(line))
23             except ValueError:
24                 pass
25
26 ts, vals = np.array(ts), np.array(vals)
27 with open("zero_drift.pkl", "wb") as f:
28     pickle.dump((ts, vals), f)
29
30 plt.figure(figsize=(8, 4))
31 plt.scatter(ts, vals, s=4, label="Filtered output")
32 plt.axhline(0, color="k", lw=0.8, ls="--")
33 plt.xlabel("Time (s)"); plt.ylabel("Weight (g)")
34 plt.title("Zero Drift -- 140 s Unloaded Recording")
35 plt.tight_layout()
36 plt.savefig("zero_drift.pdf")
```